



SZKOLNE ARKUSZE EGZAMINACYJNE

Arkusz zawiera informacje
prawnie chronione do momentu
rozpoczęcia egzaminu

Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**
Oznaczenie kwalifikacji: **INF.04**
Numer zadania: **06**

Wypełnia zdający

Numer Identyfikacyjny zdającego*

--	--	--	--	--	--	--	--

Miejsce na numer ośrodka

Czas trwania egzaminu: **180** minut.

INF.04-KG-26.06

EGZAMIN ZAWODOWY

Rok 2026

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA
2019**

Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer Identyfikacyjny.
2. Na KARCIE OCENY wpisz swoje dane:
 - swój numer Identyfikacyjny,
 - oznaczenie kwalifikacji,
 - numer zadania,
 - numer stanowiska.
3. Sprawdź, czy arkusz egzaminacyjny zawiera 7 stron i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
4. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
5. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
6. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
7. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
8. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

Powodzenia!

* w przypadku braku numeru ID – seria i numer paszportu lub innego dokumentu potwierdzającego tożsamość

Zadanie egzaminacyjne

UWAGA: katalog z rezultatami pracy oraz płytę należy opisać numerem zdającego, którym został podpisany arkusz. Dalej w zadaniu numer ten jest nazwany numerem zdającego.

Wykonaj aplikację konsolową oraz webową według wskazań. Udokumentuj obie aplikacje zrzutami ekranu. Wykorzystaj konto **Egzamin** bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsola*, *webowa*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) spakuj do archiwum. Następnie pozostaw w podfolderze jedynie pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe oraz spakowane archiwum.

Część I. Aplikacja konsolowa

Za pomocą narzędzi do tworzenia aplikacji konsolowych zaimplementuj program służący do sprawdzania czy podane 2 ciągi to anagramy.

Założenia programu

- Program wykonywany w konsoli.
- Język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym, jeden z: C++, C#, Java, Python.
- Podejście obiektowe lub strukturalne.
- Program zawiera sprawdzanie anagramu.
- Program powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.

Anagram - znaczenie

1. «wyraz utworzony przez przestawienie liter lub sylab innego wyrazu»
2. «łamigłówka, której rozwiązywanie polega na tworzeniu wyrazów przez przestawianie liter innych wyrazów»

Źródło: <https://sjp.pwn.pl/slownik-etymologiczny/anagram.html> ; dostęp: 04.02.2026

Definicja 1 - Anagram

Opis programu

Program składa się z metody/funkcji sprawdzającej czy 2 podane ciągi tekstowe są anagramami.

Założenia dotyczące kodu

- Metoda sprawdzająca posiada:
 - 2 argumenty – wyraz 1, wyraz 2
 - implementację sprawdzania anagramu
 - zwraca wartość prawda/fałsz
- Nie jest wymagana walidacja danych – zakłada się, że wprowadzono go bez błędów

Sprawdź działanie metody lub funkcji sprawdzającej. Program powinien nawiązać czytelną komunikację z użytkownikiem. Do kodu należy dołączyć testy jednostkowe opisane w części III zadania egzaminacyjnego.

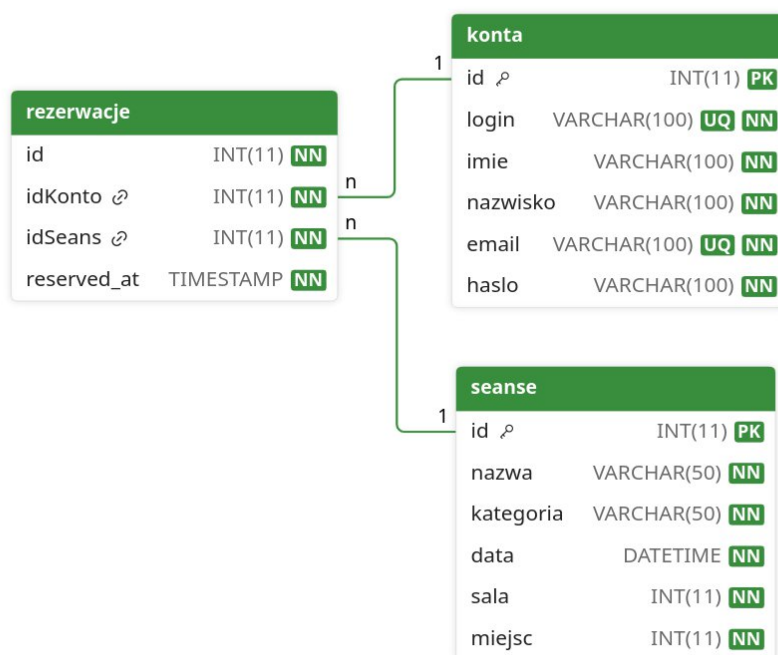
Po uruchomieniu programu wykonaj zrzuty ekranu dokumentujące interakcje programu dla różnych wyrazów. Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym. Liczba zrzutów ekranu powinna odpowiadać wszystkim możliwym interakcjom użytkownika z programem. Zrzuty ekranu zapisz w folderze konsolowa pod nazwami *konsola1.png*, *konsola2.png*, itd.

Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

Część II. Aplikacja webowa

Za pomocą środowiska programistycznego dostępnego na stanowisku egzaminacyjnym, wykonaj aplikację serwerową udostępniającą interfejs API rezerwacji kina. Do zbudowania aplikacji wykorzystaj bazę danych z archiwum **pliki6.7z** zabezpieczonego hasłem: **K1n0@26**

Podczas programowania aplikacji zmiany kodu rejestruj za pomocą repozytorium wersji Git. Po przygotowaniu projektu utwórz lokalne repozytorium wersji Git, podczas pracy utwórz co najmniej 2 zapisy wersji: po utworzeniu routerów oraz po zakończeniu aplikacji. Komentarze migawek (commit) powinny być znaczące. Skonfiguruj repozytorium Git: jako nazwę użytkownika użyj numer zdającego, jako e-mail użytkownik egzamin@poczta.pl



Obraz 1. Schemat bazy danych

Opis API:

- API składa się z 3 routerów:
 - router `/auth/` który zawiera:
 - `/login` (POST) – umożliwia logowanie użytkownika poprzez login lub e-mail oraz hasło
 - `/register` (POST) – umożliwia rejestrację użytkownika i jego zalogowanie
 - router `/reservation/` który zawiera:
 - `/reserve` (POST) - umożliwia dokonanie rezerwacji seansu przez zalogowanego użytkownika
 - `/status/<numer rezerwacji>` (GET) - zwraca informacje o rezerwacji na podstawie numeru identyfikacyjnego
 - `/room` (GET) - zwraca listę rezerwacji dla wskazanej sali i daty
 - router `/showing/` który zawiera:
 - `/` (GET) - zwraca listę wszystkich seansów
 - `/day/<dzien>` (GET) - zwraca listę seansów w wybranym dniu
 - `/details/<numer seansu>` (GET) - zwraca szczegółowe informacje o seansie

UWAGA: Po wykonaniu routerów należy zapisać stan kodu za pomocą Git

- Logowanie odbywa się przy użyciu loginu lub adresu e-mail oraz hasła. W przypadku poprawnych danych aplikacja generuje token JWT ważny przez 15 minut. Token jest zwracany w nagłówku odpowiedzi.
- Rejestracja wymaga podania loginu, imienia, nazwiska, adresu e-mail oraz hasła. Hasło powinno zostać zapisane w bazie danych w postaci zaszyfrowanej. Po poprawnym utworzeniu konta aplikacja generuje token JWT ważny 15 minut. Nie jest wymagana weryfikacja, czy użytkownik istnieje już w bazie danych.
- Rezerwacja seansu jest dostępna wyłącznie dla zalogowanych użytkowników. Token JWT powinien być przekazywany w nagłówku zapytania. Dane przekazywane do aplikacji powinny zawierać identyfikator seansu. Po poprawnym wykonaniu operacji aplikacja zwraca numer rezerwacji.
- Sprawdzenie statusu rezerwacji odbywa się na podstawie numeru rezerwacji przekazanego jako parametr zapytania. W przypadku odnalezienia rezerwacji aplikacja zwraca imię i nazwisko klienta, nazwę filmu, datę seansu oraz numer sali.
- Sprawdzenie rezerwacji dla danej sali kinowej wymaga podania poprawnego klucza autoryzacyjnego (*x-api-key*) znajdującego się w pliku zmiennych środowiskowych. Dane przekazywane do aplikacji powinny zawierać datę oraz numer sali. W przypadku odnalezienia rezerwacji aplikacja zwraca identyfikator rezerwacji, imię i nazwisko klienta oraz nazwę seansu.
- Lista wszystkich seansów powinna zawierać identyfikator seansu, nazwę filmu oraz datę i czas seansu. Dane powinny być posortowane rosnąco według daty.
- Lista seansów według dnia powinna być pobierana na podstawie daty przekazanej w formacie RRRR-MM-DD. W odpowiedzi aplikacja zwraca listę seansów odbywających się w podanym dniu, zawierającą identyfikator seansu, nazwę filmu oraz datę i godzinę seansu.
- Szczegółowe informacje o seansie powinny być zwracane na podstawie identyfikatora seansu. Dane powinny zawierać identyfikator, nazwę, kategorię, datę i godzinę seansu, numer sali oraz liczbę miejsc.

UWAGA: Po wykonaniu projektu należy zapisać jego stan za pomocą Git

Założenia aplikacji

- Aplikacja używa portu 3000
- Numer portu, dane uwierzytelniające do bazy danych oraz klucz autoryzacyjny są zapisane w pliku zmiennych środowiskowych
- Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania
- Należy stosować znaczące nazwy zmiennych, metod i kontrolek

Podjmij próbę kompilacji i uruchomienia aplikacji. Wykonaj zrzut ekranowy rezultatu z narzędzia do testowania zapytań dla seansu o numerze 4. Zrzut ekranu zapisz w folderze webowa pod nazwą: *webowa1.png*.

Kod aplikacji przygotuj do nagrania na płytę. W podfolderze webowa powinno znaleźć się archiwum całego projektu o nazwie *webowa.zip* oraz plik (lub pliki) z kodem źródłowym modyfikowanym w czasie egzaminu.

Część III. Testowanie aplikacji konsolowej i dokumentacja

Wykonaj testy jednostkowe metody/funkcji sprawdzającej anagram, utworzonej w części I zadania (aplikacja konsolowa). Do napisania testów wykorzystaj dostępną na stanowisku egzaminacyjnym bibliotekę do testów jednostkowych. Testy jednostkowe powinny sprawdzać poprawność sprawdzania anagramu.

Założenia testów

- W każdej metodzie testującej należy użyć asercji
- Każdy przypadek testowy jest realizowany w osobnej metodzie.
- Nazwy metod testujących powinny jasno określać cel testu
- Wykonuje testy z tabeli 1

Tabela 1. Scenariusz testowania z przypadkami testowymi

Nr testu	Cel testu (Jaka sytuacja będzie testowana)	Dane wejściowe	wartość
1	Sprawdzenie anagramu (podstawowy)	koł,tok	prawda
2	Sprawdzenie anagramu dla dużej wielkości liter	ala,LAA	prawda
3	Sprawdzenie anagramu dla różnej ilości znaków	dom,modl	fałsz
4	Sprawdzenie anagramu dla różnej wielkości liter	Tata,aTat	prawda

Uruchomienie testów i dokumentacja:

- Po napisaniu metod testujących uruchom wszystkie testy jednostkowe
- Wykonaj zrzut dokumentujący wyniki wykonania metod testujących
- Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym
- Zrzuty zapisz w folderze testy pod nazwami: *test1.png* itd.

Wykorzystując polecenia Git lub wbudowane narzędzia systemu kontroli wersji w środowisku programistycznym wypisz wszystkie zapisane stany kodu (migawki) wraz z komentarzami oraz z konfiguracją użytkownika Git (mail i nazwa). Wykonaj zrzut/zrzuty ekranu dokumentujące tę czynność. Zrzuty nazwij *git1.png*, *git2.png*, *git3.png*, ... i zapisz w folderze oznaczonym numerem zdającego.

Tabela 2. Selected git commands

Commands	Description
git init	Initializes a new Git repository in the current directory and creates the required repository structure.
git log	Displays the commit history of the repository, including commit identifiers, authors, dates and messages.
git add .	Adds all modified and new files from the current directory to the staging area in preparation for commit.
git commit -m "..."	Creates a new commit containing staged changes and assigns a descriptive message to the commit.
git config --global user.name "name"	Sets the global username used to identify the author of commits in all repositories on the system.
git config --global user.email "mail@example.org"	Sets the global email address associated with the commit author in all repositories on the system.

UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery: konsola, webowa, testy i plik egzamin. W folderze konsola: cały projekt aplikacji konsolowej i zrzuty. W folderze webowa: cały projekt aplikacji webowej i zrzuty. W folderze testy: cały kod testów/archiwum aplikacji testującej, ewentualnie inne przygotowane pliki. Po nagraniu płyty sprawdź poprawność nagrania. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie będą podlegać 4 rezultaty:

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja webowa,
- testy aplikacji i dokumentacja